

SQL aggregate Function and Scalar Functions.

Aggregate functions

In SQL, an **aggregate function** performs a calculation on a set of values and returns a single, summarized value. They are incredibly useful for data analysis, reporting, and getting a high-level overview of your data.

COUNT()

— — —

COUNT(): Returns the total number of rows that match a specified criterion.

```
SELECT COUNT(EmployeeID) FROM Employees;
```

SUM()

— — —

SUM(): Returns the total sum of a numeric column.

```
SELECT SUM(Salary)FROM Employees;
```

AVG()

— — —

AVG(): Returns the average value of a numeric column.

```
SELECT AVG(Salary) FROM Employees;
```

MIN()

— — —

MIN(): Returns the smallest value in a specific column.

```
SELECT MIN(Salary)FROM Employees;
```

MAX()

— — —

MAX(): Returns the largest value in a specific column.

```
SELECT MIN(Salary)FROM Employees;
```

ORDER BY

The **ORDER BY** keyword is used to sort the result-set in ascending or descending order.

The **ORDER BY** keyword sorts the result-set in ascending order (ASC) by default.

```
SELECT * FROM Products ORDER BY Price DESC;
```

GROUP BY

— — —

Aggregate functions become much more powerful when combined with the **GROUP BY** clause. While a standard aggregate function calculates a single value for the *entire* table, **GROUP BY** allows you to calculate a value for *each group* within the table. `SELECT AVG(Salary) FROM Employees GROUP BY DEPARTMENT`

Department	AverageSalary
IT	85000.00
HR	62500.00
Sales	74000.00
Marketing	68000.00

Filtering with Having

— — —
You cannot use the standard **WHERE** clause to filter the results of an aggregate function (e.g., you can't say **WHERE AVG(Salary) > 50000**). Instead, you must use the **HAVING** clause.

- **WHERE** filters rows *before* aggregation.
- **HAVING** filters groups *after* aggregation.

```
SELECT Department, AVG(Salary) AS AverageSalary  
FROM Employees  
GROUP BY Department  
HAVING AVG(Salary) > 75000;
```

SQL Scalar Functions

While aggregate functions take multiple rows and crush them down into a single summarized value, **scalar functions** do the exact opposite.

A scalar function operates on a **single value** (or a single row) and returns a **single value** based on that input. It processes your data row-by-row.

Here is a breakdown of the most common categories of scalar functions and how they are used to transform or manipulate individual pieces of data.

String Scalar Functions

UPPER() / **LOWER()**: Converts a string to all uppercase or all lowercase letters.

LENGTH() or **LEN()**: Returns the number of characters in a string.

SUBSTRING(): Extracts a specific portion of a string.

CONCAT(): Joins two or more strings together.

```
SELECT  
    FullName,  
    UPPER(FullName) AS UppercaseName,  
    LOWER(FullName) AS LowercaseName,  
    LENGTH(FullName) AS CharacterCount  
FROM TeamMembers;
```

```
SELECT  
    ID,  
    CONCAT('Employee # ', CAST(ID AS  
VARCHAR)) AS EmployeeLabel  
FROM TeamMembers;
```

Math Scalar Functions

— — —

ROUND(): Rounds a number to a specified number of decimal places.

ABS(): Returns the absolute (positive) value of a number.

CEILING() / FLOOR(): Rounds a number up (ceiling) or down (floor) to the nearest whole integer.

```
SELECT
    PerformanceScore,
    ROUND(PerformanceScore, 1) AS RoundedScore,
    ABS(PerformanceScore) AS AbsoluteScore,
    CEILING(PerformanceScore) AS RoundedUp
FROM TeamMembers;
```

Date Scalar Functions

— — —

CURRENT_DATE / GETDATE(): Returns the current system date/time.

YEAR() / **MONTH()** / **DAY()**: Extracts just the year, month, or day from a full date.

DATEDIFF(): Calculates the time difference between two specified dates.

```
SELECT
    CURRENT_DATE AS Today,
    GETDATE() AS TodayWithTime;
```

```
SELECT
    JoinDate,
    YEAR(JoinDate) AS JoinYear,
    MONTH(JoinDate) AS
JoinMonth,
    DAY(JoinDate) AS JoinDay
FROM TeamMembers;
```

Conversion Functions

Sometimes you need to change a value from one data type to another (like turning a number into text so you can combine it with a sentence).

- **CAST()**: Casts an expression of one data type to another.
- **CONVERT()**: Similar to **CAST**, but often offers specific formatting styles depending on your SQL dialect (like SQL Server).

```
SELECT  
    CONCAT('Employee #', CAST(EmployeeID AS VARCHAR)) AS EmployeeLabel  
FROM Employees;
```